Started on	Tuesday, 29 March 2022, 9:02 PM
State	Finished
Completed on	Tuesday, 29 March 2022, 9:06 PM
Time taken	4 mins 1 sec
Marks	4.00/4.00
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Mapați sursele de concurență pentru următoarele cazuri

Sistem single core, kernel nepreemptiv ✔	întreruperi
Sistem multiprocesor ✔	întreruperi, preempția cauzată de un proces mai prioritar și execuția unui proces sau întreruperi pe alt procesor
Sistem single core, kernel preemptiv ✔	întreruperi și preempția cauzată de un proces mai prioritar

Răspunsul dumneavoastră este corect.

The correct answer is: Sistem single core, kernel nepreemptiv → întreruperi, Sistem multiprocesor → întreruperi, preempția cauzată de un proces mai prioritar și execuția unui proces sau întreruperi pe alt procesor, Sistem single core, kernel preemptiv → întreruperi și preempția cauzată de un proces mai prioritar

Question **2**

Correct

Mark 1.00 out of 1.00

Completați primitivele de sincronizare necesare pentru următoarele secvențe de cod

context proces

```
static struct my_context context;
static int my_open(struct inode *inode, struct file *file)
{
    ...
    spin_lock_irqsave ✔
    // modify shared context
    spin_unlock_irqrestore ✔
    ...
}
```

rutina de tratare a unei întreruperi

```
static struct my_context context;
irqreturn_t my_handler(int irq_no, void *dev_id)
{
    ...
    spin_lock ✔
    // modify shared context
    spin_unlock ✔
    ...
}
```

Răspunsul dumneavoastră este corect.

The correct answer is:
Completați primitivele de sincronizare necesare pentru următoarele secvențe de cod

context proces

```
static struct my_context context;
static int my_open(struct inode *inode, struct file *file)
{
    ...
    [spin_lock_irqsave]
    // modify shared context
    [spin_unlock_irqrestore]
    ...
}
```

rutina de tratare a unei întreruperi

```
static struct my_context context;
irqreturn_t my_handler(int irq_no, void *dev_id)
{
    ...
    [spin_lock]
    // modify shared context
    [spin_unlock]
    ...
}
```

Question **3**

Correct

Mark 1.00 out of 1.00

Care este starea celor două linii de cache a două procesoare ce folosesc protocolul MESI pentru secvența:

Procesor A	nop	eax <- [a] (load)	nop	[a] <- eax (store)	nop
Procesor B	nop	nop	eax <- [a] (load)	nop	eax <- [a] (load)
Cache A	I	E ✔	S ✔	M ✔	S ✔
Cache B	I	I ✔	S ✔	I ✔	S ✔

Răspunsul dumneavoastră este corect.

The correct answer is:
Care este starea celor două linii de cache a două procesoare ce folosesc protocolul MESI pentru secvența:

Procesor A	nop	eax <- [a] (load)	nop	[a] <- eax (store)	nop
Procesor B	nop	nop	eax <- [a] (load)	nop	eax <- [a] (load)
Cache A	I	[E]	[S]	[M]	[S]
Cache B	I	[I]	[S]	[I]	[S]

Question **4**

Correct

Mark 1.00 out of 1.00

Completați primitivele de sincronizare RCU necesare pentru următoarele secvențe de cod

parcurgere listă

```
rcu_read_lock ✔
list_for_each_entry_rcu(i, head) {
  ...
}
rcu_read_unlock ✔
```

ștergere element din listă

```
spin_lock ✔
list_del_rcu(&node->list);
spin_unlock ✔
synchronize_rcu ✔
kfree(node);
```

; // nop

Răspunsul dumneavoastră este corect.

The correct answer is:

Completați primitivele de sincronizare RCU necesare pentru următoarele secvențe de cod

parcurgere listă

```
[rcu_read_lock]
list_for_each_entry_rcu(i, head) {
  ...
}
[rcu_read_unlock]
```

ștergere element din listă

```
[spin_lock]
list_del_rcu(&node->list);
[spin_unlock]
[synchronize_rcu]
kfree(node);
```

◄ Test Curs 4

Jump to...

Test Curs 6 ►